

Linux Foundations

Operate the system. Trust the evidence.

Ten concept → demo → lab cycles · 50% guided teaching · 50% hands-on practice



By the end, you can explain state before changing it

You will locate context, predict a result, make one bounded change, and verify it through an independent view.

- Navigate files and shell input
- Control software and access safely
- Diagnose services from evidence

Every module closes the same reasoning loop

Context → concept model → worked example → live demo → learner lab → debrief

- 50% guided concepts and demonstrations
- 50% hands-on learner incidents
- Every lab follows its prerequisite model

Predict first; type second; verify last

When stuck, capture whoami, hostname, pwd, the exact command, unchanged output, and exit status.

- Do not add sudo to silence an error
- Do not hide diagnostics with 2>/dev/null
- Reset only the assigned lab when recovery is needed

01

System boundaries and terminal survival

Which system am I operating, and how does the shell turn text into work?

General Linux + Bash · Ubuntu 24.04 lab

The operating environment has several layers

A virtual machine runs its own kernel. An Incus system container shares the controller kernel while keeping a separate user space and process view.

- Linux names the kernel; the operating environment also includes user-space tools, services, and distribution policy.
- A virtual machine runs its own kernel. An Incus system container shares the controller kernel while keeping a separate user space and process view.

The shell transforms text before execution

Bash expands variables and \$(commands), applies quoting and redirection, then starts the command and records its exit status.

- A prompt shows context; the command name selects a program or shell built-in; arguments tell it what to act on.
- Bash expands variables and \$(commands), applies quoting and redirection, then starts the command and records its exit status.

The kernel arbitrates shared resources

Applications request these services; they do not control hardware directly.

- Schedules CPU time between runnable processes
- Maps virtual memory and enforces protection
- Resolves filesystem and device operations
- Moves network data through sockets and interfaces

A system call crosses the user–kernel boundary

A command can fail in the shell, the program, or the kernel-facing operation.

- The shell starts a user-space program such as `cat`
- `cat` asks the kernel to open and read a path
- The kernel checks credentials and resolves the filesystem object
- Bytes return to `cat`, which writes them to standard output

Distribution, VM, and container describe different layers

Record both user-space and kernel facts before following external instructions.

- A distribution curates user space, packages, defaults, and support
- A virtual machine has virtual hardware and its own guest kernel
- A system container has isolated user space but shares the host kernel
- `/etc/os-release` and `uname` answer different layer questions

Read a command from left to right

```
printf 'USER=%s\nKERNEL=%s\n' \  
"$USER" "$(uname -r)" > ~/profile.txt
```

- printf receives a format plus two expanded arguments.
- Double quotes preserve each value as one argument.
- > writes standard output to a file; it replaces existing content.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

<code>\$HOME</code>	the current account's home directory
<code>\$(uname -r)</code>	capture command output
<code>> file</code>	replace a file with standard output
<code>printf</code>	produce predictable formatted text
<code>test -d PATH</code>	return success when a directory exists

Predict the evidence before Demo 1

Compare `/etc/os-release`, `uname -r`, and PID 1 on the controller and in a separate demo container.

Compare `/etc/os-release`, `uname -r`, and PID 1 on the controller and in a separate demo container.

- State the operating question
- Ask for one prediction
- Run one command at a time
- Verify through a different interface

Instructor seat only • predict before typing • verify through another interface

Before Lab 1, you should be able to...

If one item is unclear, revisit the model or worked example before opening the terminal.

- Explain kernel versus user space
- Recognize prompt, command, arguments, quoting, and expansion
- Use redirection and immediately inspect the result

System boundaries and terminal survival

Create `system-profile.txt` and `path-map.txt` that prove the seat's user space, kernel, architecture, init system, and filesystem anchors.

Create `system-profile.txt` and `path-map.txt` that prove the seat's user space, kernel, architecture, init system, and filesystem anchors.

- 15 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

The distribution and kernel answers can differ because they describe different layers. Next, paths tell us where objects live inside that environment.

The distribution and kernel answers can differ because they describe different layers. Next, paths tell us where objects live inside that environment.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

02

Filesystem navigation, objects, and links

How does Linux resolve a pathname into an object?

General Linux · FHS + Linux VFS

Paths resolve from a starting point

Absolute paths begin at /. Relative paths begin at the current working directory; every parent component must be searchable.

- Linux presents one directory namespace rooted at /. Mounted filesystems and virtual trees join that namespace at directories.
- Absolute paths begin at /. Relative paths begin at the current working directory; every parent component must be searchable.

Links connect names to objects differently

A hard link is another name for the same inode. A symbolic link stores a path and can become dangling.

- A directory entry is a name for an object. `stat` reports metadata; `file` makes a content-oriented type guess; `realpath` resolves a path.
- A hard link is another name for the same inode. A symbolic link stores a path and can become dangling.

Path resolution checks one component at a time

The final pathname can fail even when the final file mode looks permissive.

- Choose / or the working directory as the starting point
- Read each directory entry from left to right
- Follow mounts and symbolic links when encountered
- Require search permission on every parent directory

Predict link behavior before changing names

```
In notes/welcome.txt notes/welcome-hard.txt
ln -s ../notes/welcome.txt links/latest-note
ls -li notes; readlink links/latest-note
```

- Matching inode numbers prove hard-link identity.
- readlink shows the path stored in the symbolic link.
- Moving the target name breaks the symlink, not the hard link.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

<code>pwd</code>	print the current directory
<code>file PATH</code>	classify content
<code>realpath PATH</code>	show the resolved absolute path
<code>stat PATH</code>	inspect authoritative metadata
<code>find ... -printf</code>	enumerate names and link targets

Predict the evidence before Demo 2

Start in the wrong directory, let cat fail, then use pwd, realpath, namei, and stat to correct the coordinate model.

Start in the wrong directory, let cat fail, then use pwd, realpath, namei, and stat to correct the coordinate model.

- State the operating question
- Ask for one prediction
- Run one command at a time
- Verify through a different interface

Instructor seat only • predict before typing • verify through another interface

Before Lab 2, you should be able to...

If one item is unclear, revisit the model or worked example before opening the terminal.

- Distinguish absolute and relative paths
- Predict copy, move, hard-link, and symlink behavior
- Inspect exact targets before changing them

Filesystem navigation, objects, and links

Organize ~/inbox, then prove the different semantics of a copy, a hard link, and a relative symbolic link.

Organize ~/inbox, then prove the different semantics of a copy, a hard link, and a relative symbolic link.

- 25 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

Errors such as “No such file” often reveal a path assumption, not missing data. Next, the shell will turn file and log text into evidence.

Errors such as “No such file” often reveal a path assumption, not missing data. Next, the shell will turn file and log text into evidence.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

03

Shell execution, streams, editing, and text evidence

How can a pipeline turn records into a reviewable answer?

Bash + GNU text tools · Ubuntu lab

Streams separate data from diagnostics

Redirection saves a stream; it does not automatically include diagnostics. Exit status and visible output are separate evidence.

- Every process begins with standard input, standard output, and standard error. A pipe connects one command's stdout to the next command's stdin.
- Redirection saves a stream; it does not automatically include diagnostics. Exit status and visible output are separate evidence.

A pipeline is a staged data transformation

sort must precede uniq -c because uniq counts adjacent equal lines, not all equal lines anywhere in the input.

- Build pipelines one stage at a time: inspect the record grammar, select relevant records, extract a field, group adjacent values, and verify the saved result.
- sort must precede uniq -c because uniq counts adjacent equal lines, not all equal lines anywhere in the input.

A pipeline is several processes, not one command

Inspect intermediate output before adding the next stage.

- The shell creates pipes before starting the programs
- Each stage receives the previous stage's standard output
- Standard error remains separate unless explicitly redirected
- The pipeline status normally comes from its last command in Bash

Make every pipeline stage explainable

```
awk '{print $4}' access.log \  
| sort | uniq -c \  
| awk '{print $2, $1}' > status-counts.txt
```

- The first awk emits one status per line.
- sort groups equal values; uniq -c counts each group.
- The final awk normalizes the saved output to STATUS COUNT.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

nano FILE	edit interactively; Ctrl+O saves
grep PATTERN FILE	select matching records
awk	extract or reshape fields
sort	order records for comparison
uniq -c	count adjacent identical records

Predict the evidence before Demo 3

Inspect a synthetic authentication log, predict the needed fields, build the pipeline one stage at a time, then save and count the evidence.

Inspect a synthetic authentication log, predict the needed fields, build the pipeline one stage at a time, then save and count the evidence.

- State the operating question
- Ask for one prediction
- Run one command at a time
- Verify through a different interface

Instructor seat only • predict before typing • verify through another interface

Before Lab 3, you should be able to...

If one item is unclear, revisit the model or worked example before opening the terminal.

- Explain stdin, stdout, stderr, and exit status
- Describe what each pipeline stage receives and emits
- Edit, save, re-open, and validate a text file

Shell execution, streams, editing, and text evidence

Edit app.conf and reduce application and access logs into small evidence files another operator can review.

Edit app.conf and reduce application and access logs into small evidence files another operator can review.

- 20 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

A pipeline is trustworthy when each stage has a stated input and output. Next, package tools will provide several independent views of software state.

A pipeline is trustworthy when each stage has a stated input and output. Next, package tools will provide several independent views of software state.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

04

Packages and software provenance

What proves that software came from the supported package system?

Ubuntu 24.04 · APT + dpkg

Package tools expose different states

APT resolves repositories and dependencies. dpkg records installed package state and file ownership. A runnable command alone does not prove provenance.

- A repository publishes packages and signed metadata. apt update refreshes the local index; it installs no application by itself.
- APT resolves repositories and dependencies. dpkg records installed package state and file ownership. A runnable command alone does not prove provenance.

Provenance and behavior need separate proof

Use apt interactively; use apt-get for non-interactive scripts. Treat third-party repositories as a separate trust decision.

- Separate five claims: a candidate exists, the package is registered as installed, an executable is discoverable, the file belongs to that package, and the program behaves as expected.
- Use apt interactively; use apt-get for non-interactive scripts. Treat third-party repositories as a separate trust decision.

Software state has several independent layers

A command that runs is not automatically a supported package installation.

- Repository metadata says what is available
- The package database says what is registered as installed
- The shell decides which executable path it would run
- The executable's behavior is a final, separate observation

Verify software through independent views

```
apt-cache policy tree
sudo apt install tree
dpkg-query -W -f='${Status}\n' tree
command -v tree
dpkg-query -S "$(command -v tree)"
```

- Candidate state and installed state answer different questions.
- `command -v` reports shell discovery, not package ownership.
- Running `tree` verifies behavior after provenance is established.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

<code>apt-cache policy NAME</code>	inspect candidates
<code>sudo apt update</code>	refresh repository metadata
<code>sudo apt install NAME</code>	resolve and install
<code>dpkg-query -W</code>	inspect registered state
<code>dpkg-query -S PATH</code>	ask which package owns a file

Predict the evidence before Demo 4

Trace jq from repository candidate to installed state, executable path, package ownership, and JSON behavior.

Trace jq from repository candidate to installed state, executable path, package ownership, and JSON behavior.

- State the operating question
- Ask for one prediction
- Run one command at a time
- Verify through a different interface

Instructor seat only • predict before typing • verify through another interface

Before Lab 4, you should be able to...

If one item is unclear, revisit the model or worked example before opening the terminal.

- Distinguish repository, index, installed state, and files
- Explain why update and install are separate
- Verify provenance and behavior independently

Packages and software provenance

Install tree from the configured Ubuntu repositories and create evidence for candidate, state, path, ownership, and behavior.

Install tree from the configured Ubuntu repositories and create evidence for candidate, state, path, ownership, and behavior.

- 20 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

Package state is a chain of claims, not a single command result. Next, identity and permissions determine who may change those files and directories.

Package state is a chain of claims, not a single command result. Next, identity and permissions determine who may change those files and directories.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

05

Users, groups, permissions, and shared directories

Why does one identity gain access while another is denied?

General Linux DAC · Ubuntu sudo

Access checks begin with process identity

For traditional mode bits, the kernel selects one class—owner, matching group, or other. It does not add the classes together.

- The kernel authorizes numeric user and group identities. A process carries an effective user and a set of group credentials captured when its session starts.
- For traditional mode bits, the kernel selects one class—owner, matching group, or other. It does not add the classes together.

Directory permissions govern path operations

setgid on a shared directory makes new objects inherit its group. Sticky controls who may remove names in a broadly writable directory.

- On directories, r lists names, w changes names, and x traverses or searches. Access to a file requires x on every parent directory.
- setgid on a shared directory makes new objects inherit its group. Sticky controls who may remove names in a broadly writable directory.

Permission diagnosis follows a fixed decision order

Do not jump directly to chmod; first identify which check failed.

- Identify the process user and supplementary groups
- Check search permission on every parent directory
- Select owner, matching group, or other mode bits
- Test the exact read, write, create, or remove operation

Repair from identity outward

```
id alice
namei -l /srv/team-share/report.txt
sudo chown root:webteam /srv/team-share
sudo chmod 2770 /srv/team-share
sudo -u alice test -w /srv/team-share
```

- id shows the credentials used for access checks.
- namei -l exposes every path component.
- sudo -u tests the policy as the identity that matters.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

<code>id USER</code>	show UID, primary group, and supplementary groups
<code>namei -l PATH</code>	inspect every path component
<code>stat -c ... PATH</code>	report owner, group, and numeric mode
<code>sudo -u USER COMMAND</code>	test as another identity
<code>sh -c '...'</code>	run a compound command under that identity

Predict the evidence before Demo 5

Repair /srv/demo-share for demoops after inspecting two users, every path component, group ownership, and directory semantics.

Repair /srv/demo-share for demoops after inspecting two users, every path component, group ownership, and directory semantics.

- State the operating question
- Ask for one prediction
- Run one command at a time
- Verify through a different interface

Instructor seat only • predict before typing • verify through another interface

Before Lab 5, you should be able to...

If one item is unclear, revisit the model or worked example before opening the terminal.

- Select owner, group, or other without combining them
- Explain directory r, w, and x
- Test both the allowed and denied identities

Users, groups, permissions, and shared directories

Repair /srv/team-share for webteam so collaborators can create and update files while an outsider remains denied.

Repair /srv/team-share for webteam so collaborators can create and update files while an outsider remains denied.

- 30 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

Least privilege comes from matching identity, path traversal, and the intended operation. Next, SSH will authenticate both ends of a remote session.

Least privilege comes from matching identity, path traversal, and the intended operation. Next, SSH will authenticate both ends of a remote session.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

06

SSH transport, host trust, and user authentication

How does an SSH client know the server—and how does the server know the user?

OpenSSH · Ubuntu lab endpoint

SSH authenticates both ends

A host-key fingerprint must be compared through an independent trusted channel before it is recorded in `known_hosts`.

- SSH protects transport, authenticates the server with a host key, and authenticates the user separately. Encryption without verified identity is not enough.
- A host-key fingerprint must be compared through an independent trusted channel before it is recorded in `known_hosts`.

Host trust and user trust use different keys

OpenSSH checks ownership and permissions along the authorization path. A named client profile makes host, port, user, identity file, and strict host-key policy explicit.

- The user's private key stays on the client and proves possession. The authorized public key on the server permits that identity.
- OpenSSH checks ownership and permissions along the authorization path. A named client profile makes host, port, user, identity file, and strict host-key policy explicit.

An SSH connection establishes trust in stages

A failure at one stage should not be “fixed” by weakening another.

- Reach the correct host and TCP port
- Verify the server host-key fingerprint independently
- Offer proof from the client private key
- Let the server match the authorized public key and account policy

Verify host trust before user trust

```
ssh-keyscan -p 2222 training-server > host.scan  
ssh-keygen -lf host.scan -E sha256  
ssh -o BatchMode=yes training-server
```

- ssh-keyscan collects a key; it does not authenticate it.
- Compare the fingerprint with the instructor's independent value.
- BatchMode prevents a password prompt from hiding key failure.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

<code>ssh-keygen -t ed25519</code>	create a dedicated key pair
<code>ssh-keyscan</code>	collect an endpoint host key
<code>ssh-keygen -lf</code>	display a fingerprint
<code>ssh -G HOST</code>	inspect effective client configuration
<code>BatchMode=yes</code>	fail instead of prompting for a password

Predict the evidence before Demo 6

Compare the demo endpoint fingerprint through an independent source, pin it, and prove key authentication on port 2207.

Compare the demo endpoint fingerprint through an independent source, pin it, and prove key authentication on port 2207.

- State the operating question
- Ask for one prediction
- Run one command at a time
- Verify through a different interface

Instructor seat only • predict before typing • verify through another interface

Before Lab 6, you should be able to...

If one item is unclear, revisit the model or worked example before opening the terminal.

- Separate transport, server identity, and user identity
- Protect the private key and authorization path
- Diagnose network, host-key, and user-key failures separately

SSH transport, host trust, and user authentication

Create a strict client profile for training-server on port 2222, verify its host key, authorize a dedicated user key, and prove BatchMode login.

Create a strict client profile for training-server on port 2222, verify its host key, authorize a dedicated user key, and prove BatchMode login.

- 25 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

A successful SSH session rests on two explicit trust decisions. Next, logs will help reconstruct what happened when a service or login fails.

A successful SSH session rests on two explicit trust decisions. Next, logs will help reconstruct what happened when a service or login fails.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

07

Logs, journal scope, and causal events

Which event explains the failure rather than merely reporting its consequence?

systemd journal · Ubuntu 24.04

Logs are bounded event evidence

The journal stores structured fields. Scope first by unit, boot, and time; then read chronology before filtering for a suspected word.

- Logs are event evidence, not complete truth. A useful record identifies when, where, who or what, what happened, and why it matters.
- The journal stores structured fields. Scope first by unit, boot, and time; then read chronology before filtering for a suspected word.

State summaries and causal events answer different questions

The final “failed” line is often a consequence. Find the earliest causal application or manager record inside the bounded incident window.

- systemctl show provides machine-readable state such as ActiveState, Result, and ExecMainStatus. journalctl provides the event sequence that led there.
- The final “failed” line is often a consequence. Find the earliest causal application or manager record inside the bounded incident window.

A useful incident window preserves causality

Narrow scope reduces noise without destroying chronology.

- Select the affected unit and boot
- Bound the time around the reported symptom
- Read the sequence before filtering for keywords
- Separate the first cause from later failure summaries

Narrow evidence without losing chronology

```
systemctl show noisy.service \  
-p ActiveState -p Result -p ExecMainStatus  
journalctl -u noisy.service -b \  
--since "-10 min" -o short-iso --no-pager
```

- Properties summarize state; they do not explain the event sequence.
- -u and -b establish service and boot boundaries.
- Time bounds make the query repeatable and reviewable.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

<code>journalctl -u UNIT</code>	select one unit
<code>-b</code>	select the current boot
<code>--since/--until</code>	bound time
<code>-p PRIORITY</code>	filter severity after scoping
<code>--no-pager</code>	produce stable terminal output

Predict the evidence before Demo 7

Start a service that records STALE_CACHE and exits 17; identify the causal record before the later failed-state messages.

Start a service that records STALE_CACHE and exits 17; identify the causal record before the later failed-state messages.

- State the operating question
- Ask for one prediction
- Run one command at a time
- Verify through a different interface

Instructor seat only • predict before typing • verify through another interface

Before Lab 7, you should be able to...

If one item is unclear, revisit the model or worked example before opening the terminal.

- Distinguish state properties from event records
- Scope by unit, boot, and time before grepping
- Preserve the causal line and its exit evidence

Logs, journal scope, and causal events

Investigate a disk-threshold incident, preserve unit-scoped journal evidence, and explain why exit code 42 occurred.

Investigate a disk-threshold incident, preserve unit-scoped journal evidence, and explain why exit code 42 occurred.

- 20 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

The journal connects state to a causal sequence. Next, systemd declarations and relationships explain what the manager attempted to do.

The journal connects state to a causal sequence. Next, systemd declarations and relationships explain what the manager attempted to do.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

08

systemd declarations, dependencies, ordering, and diagnosis

How does systemd turn a unit declaration into runtime behavior?

systemd · Ubuntu 24.04

A unit declaration is not runtime state

Loaded, enabled, active, and successful are independent facts. status is a useful symptom summary, not a complete diagnosis.

- A unit file declares command, identity, working directory, dependencies, environment, and restart behavior. The manager tries to reach a state and records transitions.
- Loaded, enabled, active, and successful are independent facts. status is a useful symptom summary, not a complete diagnosis.

Requirement and ordering are separate relationships

After=network.target does not guarantee an application-ready network. Inspect the actual unit graph and the service's behavior instead of inferring readiness from a target name.

- Requires= and Wants= express activation and failure relationships. After= and Before= express ordering only. Ordering does not pull a missing unit into the transaction.
- After=network.target does not guarantee an application-ready network. Inspect the actual unit graph and the service's behavior instead of inferring readiness from a target name.

systemd builds a transaction before starting processes

Requires= does not mean “run first”; After= does not mean “start it”.

- Activation relationships decide which units join the transaction
- Ordering relationships decide which job runs before another
- Conditions and prerequisites can stop execution before ExecStart
- Every transition leaves manager and application evidence

Read diagnosis from declaration to behavior

```
systemctl status demo-api --no-pager  
journalctl -u demo-api -b --no-pager  
systemctl cat demo-api  
systemctl show demo-api -p Wants -p Requires -p After
```

- Status identifies the symptom and recent context.
- The journal shows transition and failure evidence.
- cat and show reveal effective declaration and relationships.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

status/is-active

observe current runtime state

is-enabled

observe boot policy

cat/show

inspect effective declaration and normalized properties

daemon-reload

reread changed unit definitions

restart

create a new execution from the effective definition

Predict the evidence before Demo 8

Diagnose demo-api.service whose required EnvironmentFile is missing; create only the missing prerequisite, restart, then verify its endpoint.

Diagnose demo-api.service whose required EnvironmentFile is missing; create only the missing prerequisite, restart, then verify its endpoint.

- State the operating question
- Ask for one prediction
- Run one command at a time
- Verify through a different interface

Instructor seat only • predict before typing • verify through another interface

Before Lab 8, you should be able to...

If one item is unclear, revisit the model or worked example before opening the terminal.

- Separate activation requirements from ordering
- Distinguish enabled, active, loaded, and successful
- Move from state to events to declaration to behavior

systemd declarations, dependencies, ordering, and diagnosis

Repair `course-web.service` from its CHDIR failure without editing the supplied unit, then prove state, enablement, socket, and HTTP behavior.

Repair `course-web.service` from its CHDIR failure without editing the supplied unit, then prove state, enablement, socket, and HTTP behavior.

- 25 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

The safest repair satisfies the declaration's missing prerequisite rather than forking it. Next, processes and sockets expose the running implementation.

The safest repair satisfies the declaration's missing prerequisite rather than forking it. Next, processes and sockets expose the running implementation.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

09

Processes, sockets, and client-visible behavior

How do unit state, process state, socket state, and useful behavior connect?

General Linux processes + iproute2 + systemd

A process is a running program with context

systemd exposes MainPID for a service. ps shows process context; /proc/PID offers supporting kernel views whose values can change while you inspect them.

- A process is a running program with a PID, parent, identity, environment, memory, and open descriptors. The executable file is only its recipe.
- systemd exposes MainPID for a service. ps shows process context; /proc/PID offers supporting kernel views whose values can change while you inspect them.

A listener is not the same as useful behavior

Use ss for the listener and a real client such as curl for behavior. Stop a managed process through systemd so manager state stays consistent.

- A listening socket proves a process is accepting connections at a transport endpoint. It does not prove that the application returns a correct response.
- Use ss for the listener and a real client such as curl for behavior. Stop a managed process through systemd so manager state stays consistent.

A service claim crosses four observable layers

Each layer can succeed while the next one still fails.

- Manager state connects the unit to MainPID
- Process evidence shows identity and command context
- Socket evidence shows whether a transport endpoint is listening
- A real client proves application-visible behavior

Trace one service through four layers

```
pid=$(systemctl show course-probe -p MainPID --value)
ps -o pid,ppid,user,stat,cmd -p "$pid"
sudo ss -ltnp 'sport = :9099'
curl --fail http://127.0.0.1:9099/
```

- MainPID connects the manager to one process.
- ss connects a listener to a port and, with privilege, a process.
- curl proves client-visible application behavior.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

`systemctl show -p MainPID`

obtain the manager's PID

`ps -p PID`

inspect one process

`/proc/PID/cmdline`

supporting argument evidence

`ss -ltnp`

inspect numeric TCP listeners

`curl --fail URL`

test behavior as a client

Predict the evidence before Demo 9

Trace demo-echo.service through MainPID, ps, command line, port 9191, and HTTP; stop it through systemd and prove the socket disappears.

Trace demo-echo.service through MainPID, ps, command line, port 9191, and HTTP; stop it through systemd and prove the socket disappears.

- State the operating question
- Ask for one prediction
- Run one command at a time
- Verify through a different interface

Instructor seat only • predict before typing • verify through another interface

Before Lab 9, you should be able to...

If one item is unclear, revisit the model or worked example before opening the terminal.

- Explain why a PID is temporary
- Separate process, listener, and application evidence
- Stop the service through its manager and verify every layer

Processes, sockets, and client-visible behavior

Trace `course-probe.service` through unit, PID, user, command line, port 9099, and HTTP, then stop it and prove the listener is gone.

Trace `course-probe.service` through unit, PID, user, command line, port 9099, and HTTP, then stop it and prove the listener is gone.

- 15 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

No single interface proves the whole service. Next, unit layering will make a configuration change without editing the supplied fragment.

No single interface proves the whole service. Next, unit layering will make a configuration change without editing the supplied fragment.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

010 Unit layering, drop-ins, reloads, and durable changes

How can an administrator override a unit without forking the vendor file?

systemd unit layering · Ubuntu 24.04

Effective unit configuration comes from layers

A drop-in records a narrow administrator override while preserving the supplied fragment for package updates and comparison. Some list-valued directives require explicit reset semantics.

- systemd assembles effective configuration from vendor fragments, administrator units, and drop-ins according to load-path precedence.
- A drop-in records a narrow administrator override while preserving the supplied fragment for package updates and comparison. Some list-valued directives require explicit reset semantics.

Reloading definitions is not restarting a process

Enabled state, active state, reloaded definitions, restarted processes, and observed behavior must be verified separately. Environment= is configuration, not a secret store.

- daemon-reload makes the manager reread unit definitions; it does not change an already running process. restart creates a new execution with the effective settings.
- Enabled state, active state, reloaded definitions, restarted processes, and observed behavior must be verified separately. Environment= is configuration, not a secret store.

A drop-in changes one layer of effective configuration

Plan removal of the drop-in as the rollback path.

- The supplied fragment remains intact
- Administrator drop-ins load with higher precedence
- daemon-reload updates the manager's definition
- restart creates a new process; behavior verifies the result

Close the loop from layer to behavior

```
systemctl show course-banner \
-p FragmentPath -p DropInPaths -p Environment
sudo systemctl daemon-reload
sudo systemctl restart course-banner
cat /run/course-banner.txt
```

- FragmentPath and DropInPaths show configuration provenance.
- daemon-reload updates the manager's definition.
- The regenerated file proves the restarted process observed the value.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

<code>systemctl cat UNIT</code>	display fragment and drop-ins
---------------------------------	-------------------------------

<code>systemctl edit UNIT</code>	create an administrator drop-in
----------------------------------	---------------------------------

<code>FragmentPath/DropInPaths</code>	inspect provenance
---------------------------------------	--------------------

<code>daemon-reload</code>	reread definitions
----------------------------	--------------------

<code>restart + behavior check</code>	apply and independently verify
---------------------------------------	--------------------------------

Predict the evidence before Demo 10

Override DEMO_LOG_LEVEL for demo-banner.service, compare effective configuration, reload, restart, and verify generated output.

Override DEMO_LOG_LEVEL for demo-banner.service, compare effective configuration, reload, restart, and verify generated output.

- State the operating question
- Ask for one prediction
- Run one command at a time
- Verify through a different interface

Instructor seat only • predict before typing • verify through another interface

Before Lab 10, you should be able to...

If one item is unclear, revisit the model or worked example before opening the terminal.

- Explain fragment and drop-in precedence
- Distinguish reload of definitions from restart of a process
- Design verification and rollback before changing configuration

Unit layering, drop-ins, reloads, and durable changes

Set `COURSE_MODE=production` with a drop-in, reload definitions, restart the service, and prove effective configuration and regenerated behavior.

Set `COURSE_MODE=production` with a drop-in, reload definitions, restart the service, and prove effective configuration and regenerated behavior.

- 15 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

A durable change is complete only when provenance, runtime state, and behavior agree—and rollback is understood.

A durable change is complete only when provenance, runtime state, and behavior agree—and rollback is understood.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

A safe operator closes every change with evidence

Locate context → predict state → act narrowly → observe output and events → verify behavior independently.

- Declaration is not runtime state
- Runtime state is not useful behavior
- One successful command is not the whole claim

Explain the chain when you are blocked

A useful help request contains enough unchanged evidence for someone else to reason with you.

- Which user, host, and working directory?
- What exact target state did you inspect?
- What command and unchanged output can be repeated?
- What changed, and what independent check followed?

Protect the 50/50 balance when adapting delivery

If the room schedule changes, shorten examples and discussion evenly; keep all ten labs, verification steps, and debriefs.

- 50% guided teaching
- 50% hands-on practice
- Do not trade learner verification for lecture time

Practise the loop on a reset incident

Reset one lab, solve it without copying the prior command sequence, and explain the evidence aloud.

- Rebuild the SSH profile from the trust model
- Explain 640 and 2770 without a lookup
- Trace a service from unit to client behavior

Primary manuals define the claims

The course claim ledger maps every module and lab to authoritative documentation.

- GNU Bash Reference Manual
- Ubuntu Server documentation
- systemd manuals
- OpenSSH manuals
- Linux man-pages and FHS
- Incus instance documentation

Operate the system. Trust the evidence.

You do not need to remember every command. You need to know what question to ask, what changed, and what would prove it.
